Boneh-Franklin IBE scheme with Optimized r-Ate Pairing over BLS12-381 curve

Cryptographic Computing
Group 2 Project

Miklos Daniel Vadasz
202402632@post.au.dk

Aarhus University

December 2, 2025

Outline

Boneh-Franklin IBE scheme Overview

Identity-based Encryption

- Alternative to PKI
- Encryption key generated from some Identity (ID)
- Parameters shared with Trust Center (TPG)
  - That both *Alice* and *Bob* trusts
- Sender has to use only ID to encrypt messages

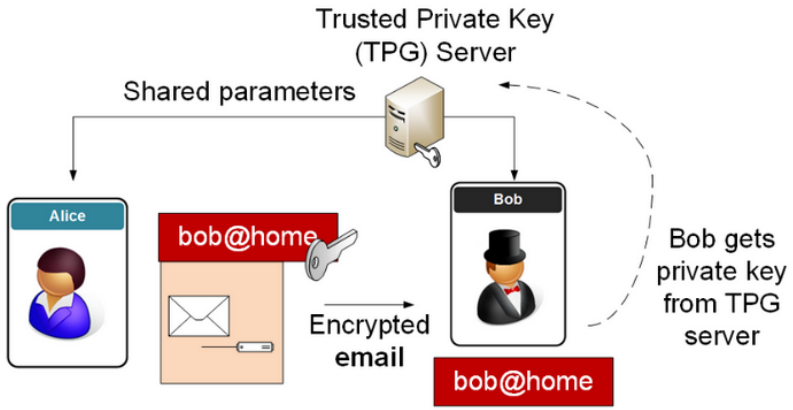


Figure: BF-IBE scheme overview.[1]

Boneh-Franklin IBE scheme Core Algorithms

- **Setup:**
  - Input: k security parameter
  - Output: system parameters, master-key
- **Extract:**
  - Input: system parameters, master-key as $\mathrm{ID} \in 0, 1^*$
  - Output: d private-key
- **Encrypt:**
  - Input: system parameters, ID and $M \in \mathcal{M}$
  - Output: $C \in \mathcal{C}$
- **Decrypt:**
  - Input: system parameters, private-key, $C \in \mathcal{C}$
  - Output: $M \in \mathcal{M}$

Setup phase: ;
Input: k, security parameter chosen from $\mathbb{Z}^+$
Output: System parameters params $= \langle q, \mathbb{G}_1, \mathbb{G}_2, \hat{e}, n, P, P_{pub}, H_1, H_2 \rangle$
Step 1: We run $\mathcal{G}$ on k which yields a prime q, $\mathbb{G}_1, \mathbb{G}_2$ groups of order q, and an admissible bilinear map $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}$.Random generator is chosen $P \in \mathbb{G}_1$.
Step 2: We pick a random $s \in \mathbb{Z}_q^*$ then set $P_{pub} = sP$, where s is the master-key $s \in \mathbb{Z}_q^*$
Step 3: Cryptographic hash H_1, H_2, H_3, H_4 functions are chosen as random oracles:
$H_1 : \{0,1\}^* \to \mathbb{G}_1^*$
$H_2 : \mathbb{G}_2 \to \{0,1\}^n$ for some n.
$H_3 : \{0,1\}^n \times \{0,1\}^n \to \mathbb{Z}_q^*$
$H_4 : \{0,1\}^n \to \{0,1\}^n$
Extract phase: ;
For given ID $\in \{0,1\}^*$ string we compute $Q_{id} = H_1(ID) \in \mathbb{G}_1^*$ and setting d_{id} private key to be $d_{ID} = sQ_{ID}$, where s is the master key as before.
Encrypt phase: ;
To encrypt $M \in \{0,1\}^n$ with public key ID, we compute: $Q_{ID} = H_1(ID) \in \mathbb{G}_1^*$, then choose a random $\sigma \in \{0,1\}^n$, then set $r = H_3(\sigma, M)$ lastly the C ciphertext can be computed as: $C = \langle rP, \sigma \oplus H_2(g_{ID}^r), M \oplus H_4(\sigma) \rangle$, where $g_{ID} = \hat{e}(Q_{ID}, P_{pub}) \in \mathbb{G}_2$.
Decrypt phase: ;
Let $C = \langle U, V, W \rangle$ be the ciphertext encrypted with public key ID. First we check $U \neq \mathbb{G}_1^*$ holds, if true we reject the C ciphertext. To decrypt C with private key $d_{ID} \in \mathbb{G}_1^*$ we proceed as follows:
1. Compute $V \oplus H_2(\hat{e}(d_{ID}, U)) = \sigma$
2. Compute $W \oplus H_4(\sigma) = M$.
3. Set $r = H_3(\sigma, M)$. Test that $U = rP$, if not reject the ciphertext.
4. Output M as the decryption of C.

Algorithm 1: Boneh-Franklin based scheme: FullIdent Algorithm

Figure: Formal definition of the four core algorithms

Bilinear mapping and Diffie-Hellmann assumption

- **Bilinear mapping:**
  - $\mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$

- **Properties must be satisfied:**
  - Bilinearity:

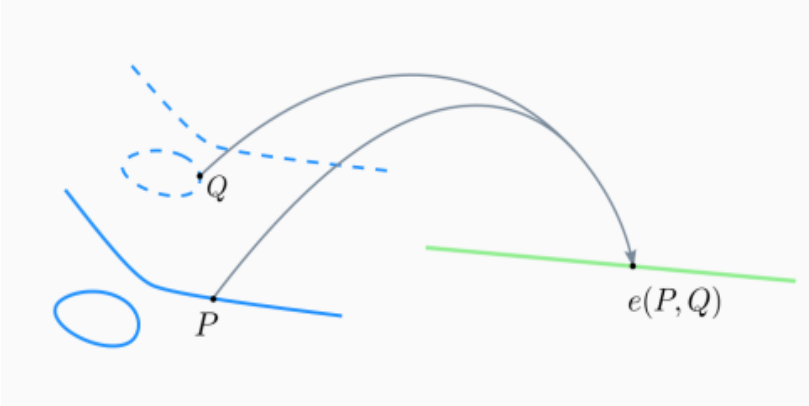
    $$\hat{e}(aP, bQ) = \hat{e}(P, Q)^{ab}$$

  - Non-degeneracy:

    $\hat{e}$ does not send all pairs $\mathbb{G}_1 \times \mathbb{G}_1$ to $\mathbb{G}_2$

  - Computability:

    $\exists$ Some efficient algorithms to compute $\hat{e}(P, Q)$ for any $P, Q \in \mathbb{G}_1$



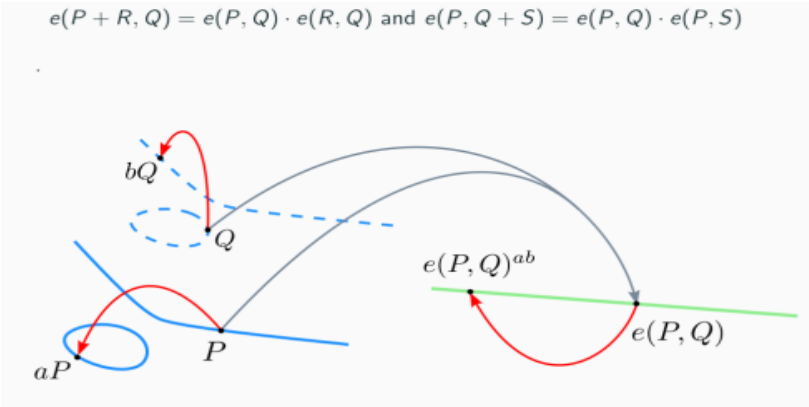


Figure: Pairings over elliptic curves[2].

Bilinear mapping and Diffie-Hellmann assumption[2]

- **Classical problems:**
  - *Discrete Log Problem (DLP) :*

    Recover a from $\langle g, g^a \rangle$
  - *Computational Diffie-Hellman Problem (CDHP):*

    Compute g^{ab} from $\left\langle g, g^a, b^b \right\rangle$
- **Underlying problems:**
  - *Elliptic Curve Discrete Log Problem (ECDLP):*

    Recover a from $\langle P, aP \rangle$
  - *Bilinear Computational Diffie-Hellman Problem (BCDHP):*

    Compute $e(P, Q)^{abc}$ from $\langle P, aP, bP, cP, Q, aQ, bQ, cQ \rangle$

Why elliptic curves ?

- Some properties we can rely on[2]:

  Underlying problem harder than integer factoring (RSA)

  Same security level with smaller parameters

  Efficient storage and execution time

Weierstrass form:

$$y^2 = x^3 + ax + b$$

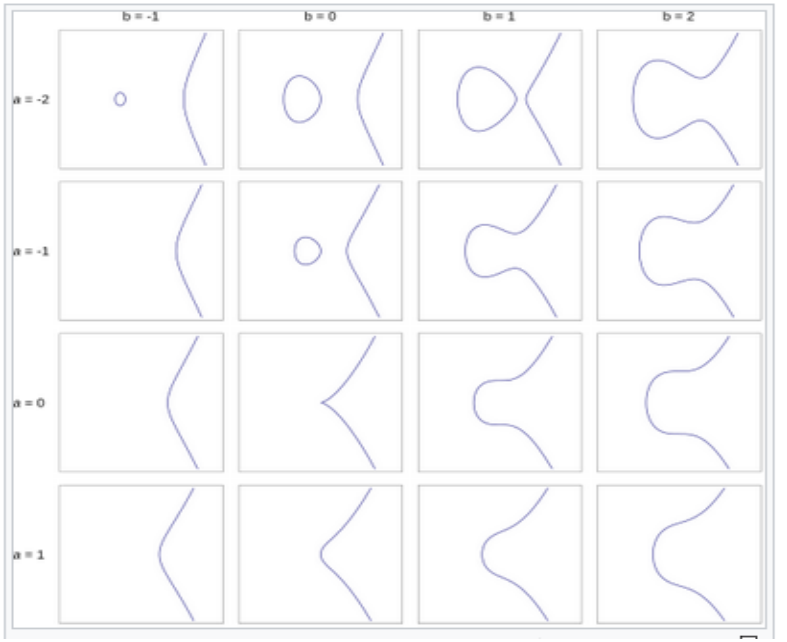


Figure: Elliptic curves with different system parameters.[3]

Pairing-friendly curve: BLS12-381

- **Subgroups:**

  $\mathbb{G}_1$ over $\mathbb{F}_p$,

  $\mathbb{G}_2$ over $\mathbb{F}_{p^2}$

- **Target group:**

  $\mathbb{G}_T \subset \mathbb{F}_{p^{12}}$

- **Pairing:**

  $e([a]P, [b]Q) = e(P, [b]Q)^a = e(P, Q)^{ab}$

  $P \in \mathbb{G}_1$,

  $Q \in \mathbb{G}_2$

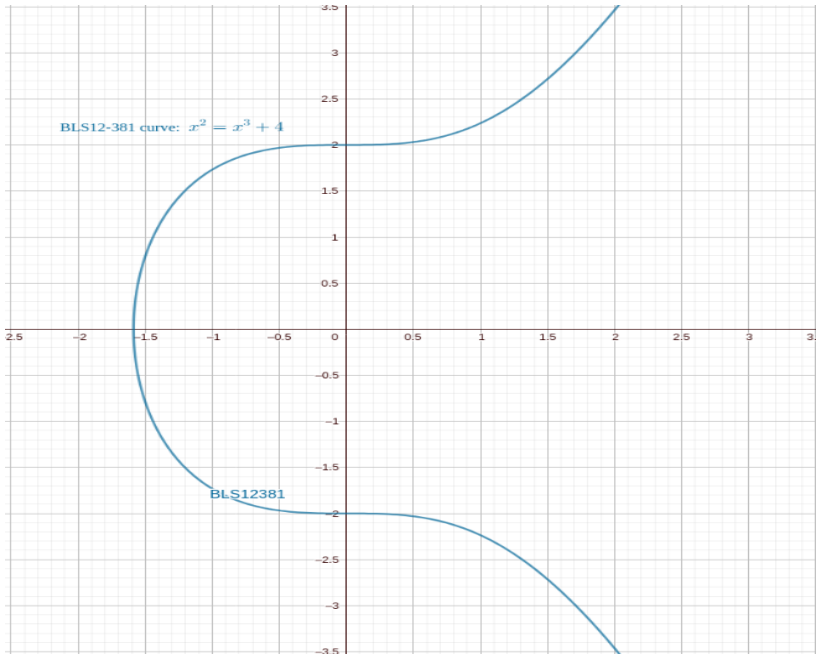


Figure: Plotting BLS12-381 curve.
Equation: $y^2 = x^3 + 4$

Pairing Computation

Miller's Algorithm

Algorithm 5 Tate pairing [Barreto et al. 2002].

Input: $r = \sum_{i=0}^{\log_2 r} r_i 2^i, P, Q$.
Output: $e_r(P, Q)$.

1: $T \leftarrow P$
2: $f \leftarrow 1$
3: $s \leftarrow r - 1$
4: **for** $i = \lfloor \log_2(s) \rfloor - 1$ **downto** 0 **do**
5: $\quad T \leftarrow 2T$
6: $\quad f \leftarrow f^2 \cdot l_{T,T}(Q)$
7: $\quad$ **if** $s_i = 1$ **then**
8: $\quad\quad T \leftarrow T + P$
9: $\quad\quad f \leftarrow f \cdot l_{T,P}(Q)$
10: $\quad$ **end if**
11: **end for**
12: **return** $f^{(q^k-1/r)}$

Figure: Miller's algorithm for Tate pairing. [2]

```
/* Optimal R-ate pairing */
public static FP12 ate(ECP2 P1, ECP Q1) {
    FP2 f;
    BIG n = new BIG(0);
    BIG n3 = new BIG(0);
    ECP2 K = new ECP2();
    FP12 lv, lv2;
    int bt;

    if (Q1.is_infinity()) return new FP12(1);
// P is needed in affine form for line function, Q for (Qx,Qy) extraction
    ECP2 P = new ECP2(P1);
    ECP Q = new ECP(Q1);

    P.affine();
    Q.affine();
```

Figure: Code snippet from MIRACL[4] PAIR.java class

Some challenges we faced

- *jPBC*[5]: outdated library, BLS curve is not supported at all, BN curve properties has to be "hardcoded"
- *Apache Milagro*[6]: fairly outdated library, BLS curve is supported, but $\mathrm{mapit}()$ does not use *SWU* method

  Refer to implementation under: BF_IBE__BLS12381_TryAndIncrement/

- *MIRACL*[4]: updated library,BLS curve fully supported, encodig and hashing into the curve according to RFC9380

  Refer to implementation under: BF_IBE__BLS12381_MIRACL/

Hashing/encoding into BLS12-381 Elliptic Curve

- Two distinct algorithms:
  - encode_to_curve()
  - hash_to_curve()
- Hash-to-curve problem:
  - "Try-and-Increment" method (not recommended)
  - *SWU* algorithm (recommended)

Input: msg, an arbitrary-length byte string
Output: P, a point in G
Steps: ;
1. $u = \text{hash\_to\_field}(msg, 1)$;
2. $Q = \text{map\_to\_curve}(u[0])$;
3. $P = \text{clear\_cofactor}(Q)$;
4. return P;

Algorithm 3: Pseudocode for encoding into elliptic curve[19] under Random-Oracle Model

Input: msg, an arbitrary-length byte string
Output: P, a point in G
Steps: ;
1. $u = \text{hash\_to\_field}(msg, 2)$;
2. $Q0 = \text{map\_to\_curve}(u[0])$;
3. $Q1 = \text{map\_to\_curve}(u[1])$;
4. $R = Q0 + Q1$;
5. $P = \text{clear\_cofactor}(R)$;
6. return P;

Algorithm 4: Pseudocode for hashing to elliptic curve[19] under Random-Oracle Model

Figure: Pseudocode of encode_to_curve() and hash_to_curve() algorithms[7]

Test results

```
======================================================================
TESTING COMPLETE ENCRYPT/DECRYPT CYCLE (MIRACL-CORE)
======================================================================

=== Extract Algorithm ===
Generating private key for identity: bob@companyEU.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (009e9db06e3dcc7934d088bf37b8d78c05cce8ca1f24b91d4b38145f02b81e968fd14df98e76d152d4a88429a3852a4
Step 2: d_ID = s * Q_ID computed
d_ID: (0bfec4e602cb1303d4fa34404dc33736ce4cf5634839585efff052157ebe2dcd95245504d3a09556c5f525bd297c963
Extract complete - Private key generated for: bob@companyEU.com

=== Extract Algorithm ===
Generating private key for identity: alice@companyUS.com
Step 1: Q_ID = H1(ID) computed
Q_ID: (011c62e5b06ea5ee526850332343744 0a94b54faaeea3c01207c5c7f75adf71725ea0e3dec1ca13b7a82d0239079230
Step 2: d_ID = s * Q_ID computed
d_ID: (0293ae1a38eb96a07ea3592055098b6c97b4fede6287043c732ca73bdec726a1b9ca84e8336f22c7ff3150bb4e7b7ff
Extract complete - Private key generated for: alice@companyUS.com

======================================================================
--- Test 1: Basic Encrypt/Decrypt ---
======================================================================
Original message: "Hello Bob! This is a secret message."
Message length: 36 bytes
Message (hex): 48656c6c6f20426f6221205468697320697320612073656372657420 6d6573736167652e

=== Encrypt Algorithm ===
Encrypting for: bob@companyEU.com
Encryption complete

--- Ciphertext Components ---
U (G1 element): (0d769ff61e77db14280a7c80e80c1a10c809d64239a0b3c1ec1e5d6fa66aeba2e505656a46e72710894a5
V (32 bytes): 339123a061ba7e522ca1cab07bd822e4eab7559a01b15d300312d23c55bafa9b
W (36 bytes): e87be41d5cf9fe3f6b8849db58a2e0258b9de90b0f58f90f7f9f3c5fb78e41fb6167652e
Total ciphertext size: 173 bytes

=== Decrypt Algorithm ===
Decrypting for: bob@companyEU.com
Decryption successful

--- Decryption Result ---
Decrypted message: "Hello Bob! This is a secret message."
Decrypted (hex): 48656c6c6f20426f6221205468697320697320612073656372657420 6d6573736167652e

Messages match: true
Test 1:  PASS
```

Figure: Functionality tests for BF-IBE scheme

Further research

- Master-key generation with *SSS* plus *VSS*
  - *FROST* [8] protocol ?
  - CSI-RAShi[9] ?
  - etc.
- Different IBE constructions based on:
  - Pairings,
  - Lattices,
  - Quadratic residuosity

IBE-based constructions			
	Based on pairings, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$	**Based on lattices** (Learning with errors)	**Quadratic Residuosity**
IBE with *Random Oracle*	Boneh-Franklin scheme[9],Boneh-Boyek scheme[7] , Waters-IBE[32] , Gentry-IBE[22] , DualSys-IBE[**W09**]	Trapdoors for hard lattices[23]	Space-efficient IBE[10]
IBE without *Random Oracle*	Forward-secure PKE[14]	Delegation of lattice basis[16]	*future work*
Hierarchical IBE	Hierarchical ID-based cryptography[24]	Delegation of lattice basis[16]	*future work*

Figure: Overview of Identity-Based cryptographic construction families[10]

References I

[1] William J Buchanan. *Identity-based Encryption (IBE)*. https://asecuritysite.com/encryption/ibe. Accessed: October 25, 2025. Asecuritysite.com, 2025. URL: https://asecuritysite.com/encryption/ibe.

[2] Diego F. Aranha. *Introduction to Pairings*. [Online; accessed 25. May. 2025]. 2017. URL: https://ecc2017.cs.ru.nl/slides/ecc2017school-aranha.pdf.

[3] *Elliptic curve*. [Online; accessed 25. Oct. 2025]. URL: https://en.wikipedia.org/wiki/Elliptic_curve.

[4] *The MIRACL Core Cryptographic Library*. [Online; accessed 13. November. 2025]. URL: https://github.com/miracl/core/tree/master/java.

References II

[5] Angelo De Caro and Vincenzo Iovino. "jPBC: Java pairing based cryptography". In: *2011 IEEE Symposium on Computers and Communications (ISCC)*. 2011, pp. 850–855. DOI: 10.1109/ISCC.2011.5983948.

[6] *Apache Milagro*. [Online; accessed 05. Nov. 2025]. 2017. URL: https://github.com/apache/incubator-milagro.

[7] Armando Faz-Hernandez et al. *Hashing to Elliptic Curves*. RFC 9380. Aug. 2023. DOI: 10.17487/RFC9380. URL: https://www.rfc-editor.org/info/rfc9380.

[8] Chelsea Komlo and Ian Goldberg. *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. Cryptology ePrint Archive, Paper 2020/852. 2020. URL: https://eprint.iacr.org/2020/852.

[9] Ward Beullens et al. *CSI-RAShi: Distributed key generation for CSIDH*. Cryptology ePrint Archive, Paper 2020/1323. 2020. URL: https://eprint.iacr.org/2020/1323.

References III

[10] Dan Boneh and Matthew Franklin. "Identity-Based Encryption from the Weil Pairing". In: *SIAM Journal on Computing* 32.3 (2003), pp. 586–615. DOI: 10.1137/S009753970139852l. eprint: https://doi.org/10.1137/S009753970139852l. URL: https://doi.org/10.1137/S009753970139852l.